



Volume Number: 14 (1998)
Issue Number: 2
Column Tag: Programmer's Challenge

Programmer's Challenge

by Bob Boonstra, Westford, MA

Image Decoder

This month's Challenge is much easier to state than it will be to implement. Your task this month is to write a decoder for Graphics Interchange Format images. GIF is a data stream-oriented file format used to define the transmission protocol of LZW-encoded bitmap data. Your code will read a file containing a GIF and draw the image in an offscreen graphics world. There are two versions of the GIF specification; this Challenge includes only the earlier and more common GIF87a format, not the GIF89a format. The format description is too long to include here, but you can find the GIF87a specification at:
<ftp://ftp.ncsa.uiuc.edu/misc/file.formats/graphics.formats/gif87a.doc>

Other useful information on graphics formats in general is available at: <http://www.cis.ohio-state.edu/hypertext/faq/usenet/graphics/fileformats-faq/top.html>

The prototype for the code you should write is:

```
#include <QDOffscreen.h>
#include <stdio.h>
GWorldPtr ReadImage(FILE *inputFile);
```

The inputFile will have been opened for you in binary mode by the calling routine. ReadImage should read and parse the image, create a GWorld with the appropriate color table, draw the image in the GWorld, and return a GWorld pointer to the calling routine. The solution that correctly decodes a variety of images in the least amount of execution time will be the winner.

I thought for some time about the Compuserve / Unisys / LZW patent controversy before selecting this topic for the Challenge. The LZW algorithm used in GIF compression is patented by Unisys, and Unisys requires that any commercial software product that uses this algorithm, including software that reads GIF images, requires a license from Unisys. Freely distributed software does not require a license. Since the Challenge doesn't result in a software product, much less a commercial product, I decided to go ahead with this exploration of decoding efficiency.

This will be a native PowerPC Challenge, using the latest CodeWarrior environment. Solutions must be coded in C, C++, or Pascal. This Challenge was suggested by Peter Lewis, who earns 2 Challenge points for the suggestion.

Three Months Ago Winner

Congratulations to **Jeff Mallett** (Boulder Creek, CA) for submitting the winning entry to the Pente(tm) Challenge. The objective was to accumulate, in minimum execution time, the most points in a round-robin tournament of the board game Pente(tm). With only three entries, the tournament required only six games for each player to compete against each other player twice, once playing first and once playing second. Both the winning entry and the third-place entry by Randy Boring employed an alpha-beta search technique to find the best move, and both entries were successful in winning most of their games. Jeff's entry, however, was more efficient about pruning the search tree and therefore required significantly less execution time than Randy's entry. Since a point was deducted for each second of execution time, the winning entry lost significantly fewer points for inefficiency. In addition, the scoring algorithm in Jeff's AddStone routine takes advantage of all three

Showcase

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to PROACTIVE support and be the superhero your users expect

WATCHMAN
MONITORING



THIS MONTH'S ISSUE



Issue 353

Code Debugging and
Spelunking

Check out the latest including:
 x86 Code Debugging and
 Spelunking ; An Introduction
 to WebGL ; Managing ESXi
 from your Mac ; Deploying
 SHA-3 ; An introduction to
 Julia ; KoolTools: BBEEdit 11.5,
 TextSoap For OS X, Honeywell
 Lyric Round Wi-Fi Thermostat,
 My Cloud EX2 Ultra ; MacTech
 Spotlight: Daniel H Steinberg

See Now



Remember
when typing
felt good?



tactilepro
the satisfying click of
mechanical keyswitches



Matias
Folding Keyboard
full-size & portable keyboard
for iPhone, iPad, & Mac

matias

TODAY'S DEAL

methods of scoring points: captures, winning by placing 5 stones in a row, and leaving sequences of 4-in-a-row on the board at the conclusion of a game. His solution earned more points as a result, even though it won the same number of games as Randy's.

The table below lists the number of tournament wins, points earned, cumulative score, code size, data size, and programming language for each entry. The number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges to date prior to this one.

Name	Wins	Points	Score	Code	Data	Language
Jeff Mallett (74)	3	33	32.03	4872	7923	C
Sebastian Maurer	0	10	9.97	4860	208	C++
Randy Boring (66)	3	23	-374.90	7672	2649	C

Top 20 Contestants

Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants. The scores have been adjusted to correct an error made in assigning points for the Stratego Challenge printed in the November issue.

Rank	Name	Points
1.	Munter, Ernst	200
2.	Boring, Randy	73
3.	Cooper, Greg	61
4.	Lewis, Peter	59
5.	Mallett, Jeff	50
6.	Nicolle, Ludovic	48
7.	Murphy, ACC	34
8.	Gregg, Xan	33
9.	Antoniewicz, Andy	24
10.	Day, Mark	20
11.	Higgins, Charles	20
12.	Larsson, Gustav	20
13.	Studer, Thomas	20
14.	Gundrum, Eric	15
15.	Hart, Alan	14
16.	O'Connor, Turlough	14
17.	Picao, Miguel Cruz	14
18.	Saxton, Tom	12
19.	Cutts, Kevin	11
20.	8-way tie	10

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points
2nd place	10 points
3rd place	7 points
4th place	4 points
5th place	2 points
finding bug	2 points
suggesting Challenge	2 points

Here is Jeff's winning solution:

```
MyPente.c
Copyright © 1997 Jeff Mallett
//
// Do a depth=1 search to sort moves. Then do a higher
// depth alpha-beta search to select the best move.
//
// Plays for score rather than to win. For example,
// doesn't try to win by getting 5 captures since
// there is no score incentive for this. It could be
// retuned to play for a win instead.

#include "Pente.h"

#define SEARCH_DEPTH 4
enum {
    _FRIEND = 0x0001,
    _ENEMY = 0x0002,
    _EMPTY = 0x0004,
    _WALL = 0x0008
```

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to *PROACTIVE* support and be the superhero your users expect



Search

Community Search:

MacTech Search:

Software Updates via MacUpdate



Remember
when typing
felt good?

Latest Forum Discussions

iPHONE **APP** SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

```

};

#define BORDER      1
#define BORDERS     2
#define INFINITY    32600
#define NA          0
#define MAX_CELLS   (31 + BORDERS) * (31 + BORDERS)

#define ESTIMATE_PLUS 1
#define WIN          2000
#define CAPTURE_SCORE 240
#define VULNERABLE   150

static short CHAIN_SCORE2[3][3] = { //[color][open]
    { NA, NA, NA },
    { -1, -9, -2 }, // FRIEND
    { 1, 9, 2 } // ENEMY
};
static short CHAIN_SCORE3[3][3] = { //[color][open]
    { NA, NA, NA },
    { 3, 16, 40 }, // FRIEND
    { -2, -10, -30 } // ENEMY
};
static short CHAIN_SCORE4[3][3] = { //[color][open]
    { NA, NA, NA },
    { 230, 240, 250 }, // FRIEND
    { -150, -155, -160 } // ENEMY
};
static short CHAIN_SCORE5[3] = { //[color]
    NA, WIN, -WIN
};
//          1 2 3 4
static short BLOCK_SCORE[5] = { NA, 0, 14, 14, 22};
static short THREATS[] = {
    NA, NA,
    25, // 2: tria
    30, // 3: half-open four
    350, // 4: double trias
    370, // 5: tria + half-open four
    450, // 6: tessera or 2 half-open fours
    500, 500, 500, 500, 500, 500, 500, 500, 500, 500
};

static short gPreScore[MAX_CELLS], gEstimates[MAX_CELLS];
static short *gEstimatesStart, *gEstimatesEnd;
static short gBoard[MAX_CELLS], *gBoardStart, *gBoardEnd;
static short *gFirstStone, *gLastStone;
static short *gKillers[SEARCH_DEPTH], gPreviousThreats;
static short gDirections[8], *gDirectionsEnd;
static short gMoveNum, gScore, gStartDepth;
static short gBoardHalfSize, gSideLength, gBoardMax;
static short gCumCapturesFriend, gCumCapturesEnemy;
static short gAdjustment, gSE;

#define ADJUST(a)    ( (a) + gAdjustment )
#define TRANSLATE(x, y) \
    (gSideLength * ADJUST(y) + ADJUST(x))
#define GET_INDEX(p) ((p) - gBoard)
#define GET_X(i)    ( ((i) % gSideLength) - gAdjustment )
#define GET_Y(i)    ( ((i) / gSideLength) - gAdjustment )
#define UPDATE_ENDPOINTS(pSq, a, b) \
    if (pSq < a) a = pSq; \
    if (pSq > b) b = pSq

#define OPPONENT(side) (3 - (side))
#define OCCUPIED(x)    ((x) & (_FRIEND | _ENEMY))
#define EMPTY(x)       ((x) == _EMPTY)

// gChanges -- Array of unsigned longs containing data to
// undo moves
// list of:
// <pointer to position> <old position value>
// terminated by a 0L

```

```

static unsigned long gChanges[256], *gChangesEnd;
#define PUSH(x)      *(gChangesEnd++) = (x)
#define START_SAVE  PUSH(0L)
#define PUSH_SQ(pSq) \
    { PUSH((long)*(pSq)); PUSH((unsigned long)(pSq)); }
#define POP          * (--gChangesEnd)
#define TOP          *gChangesEnd

static short * ChooseNextMove();
static short AddStone(short alpha, short beta, short *pSq,
    short color, short depth,
    short capturesFriend, short capturesEnemy,
    short *firstStone, short *lastStone);
static long MyFindCaptures(Capture capture[], short *pSq,
    short us, short them);
static Boolean MyFindFive(short *pSq, short color);

InitPente
void InitPente(
    long boardHalfSize /* e.g., 9 for a 19x19 board */
    /* all coordinates between -boardHalfSize
    and +boardHalfSize */
) {
    short i, j, *pSq;

    gBoardHalfSize = boardHalfSize;
    gSideLength = boardHalfSize * 2 + 1 + BORDERS;
    gSE = gSideLength + 1;
    gDirections[0] = -gSE;          // NW
    gDirections[1] = -gSideLength; // N
    gDirections[2] = -gSideLength + 1; // NE
    gDirections[3] = -1;           // W
    gDirections[4] = gSE;          // SE
    gDirections[5] = gSideLength;  // S
    gDirections[6] = gSideLength - 1; // SW
    gDirections[7] = 1;            // E
    gDirectionsEnd = &gDirections[8];

    gBoardMax = gSideLength * gSideLength;
    i = (gSideLength + 1) * BORDER;
    gBoardStart = &gBoard[i];
    gBoardEnd = &gBoard[gBoardMax - i];
    gEstimatesStart = &gEstimates[i];
    gEstimatesEnd = &gEstimates[gBoardMax - i];
    gFirstStone = gBoardEnd;
    gLastStone = gBoardStart;

    pSq = gBoard;
    do {
        *pSq = _WALL;
    } while (++pSq != gBoardStart);
    do {
        *pSq = _EMPTY;
    } while (++pSq != gBoardEnd);
    do {
        *pSq = _WALL;
    } while (++pSq < &gBoard[gBoardMax]);
    for (i = BORDER + boardHalfSize * 2 + 1;
        i < gBoardMax - gSideLength; i += gSideLength)
        for (j = 0; j < BORDERS; ++j)
            gBoard[i + j] = _WALL;
    gCumCapturesFriend = gCumCapturesEnemy = gMoveNum = 0;
    gAdjustment = gBoardHalfSize + BORDER;
    gStartDepth = SEARCH_DEPTH - 1;
}

Pente
void Pente(
    Point opponentsMove, /* your opponent moved here */
    Boolean playingFirst, /* ignore opponentMove */
    Point *yourMove, /* return your move here */
    Capture claimCaptures[], /* return captured pairs here */
    long *numCaptures, /* return # of claimCaptures */

```

```

Boolean *claimVictory      /* return true if you claim
                           victory with this move */
) {
    Point move;
    Capture opponentCaptures[8];

    short *pSq, *pNewSq, *d;
    short score, bestScore, i;
    short *pBestMove;

    *numCaptures = 0;
    *claimVictory = false;
    if (playingFirst) { // *** MOVE 1
        move.h = move.v = 0;
        pSq = &gBoard[TRANSLATE(0, 0)];
        *pSq = _FRIEND;
        UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
        *yourMove = move;
        gMoveNum = 1;
        return;
    }

    i = TRANSLATE(opponentsMove.h, opponentsMove.v);
    pSq = &gBoard[i];
    *pSq = _ENEMY;
    UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
    ++gMoveNum;

    if (gMoveNum == 1) { // *** MOVE 2
        move.h = move.v = -2;
        pSq = &gBoard[TRANSLATE(-2, -2)];
        *pSq = _FRIEND;
        UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
        *yourMove = move;
        gMoveNum = 2;
        return;
    }

    if (gMoveNum == 2) { // *** MOVE 3
        if (opponentsMove.v < opponentsMove.h) {
            if (opponentsMove.v < -opponentsMove.h) {
                // top triangle
                move.h = 0;
                move.v = 4;
            } else {
                // right triangle
                move.h = -4;
                move.v = 0;
            }
        } else {
            if (opponentsMove.v >= -opponentsMove.h) {
                // bottom triangle
                move.h = 0;
                move.v = -4;
            } else {
                // left triangle
                move.h = 4;
                move.v = 0;
            }
        }
        pSq = &gBoard[TRANSLATE(move.h, move.v)];
        *pSq = _FRIEND;
        UPDATE_ENDPOINTS(pSq, gFirstStone, gLastStone);
        *yourMove = move;
        gMoveNum = 3;
        return;
    }

    gChangesEnd = gChanges;
    gCumCapturesEnemy +=
        MyFindCaptures(opponentCaptures, &gBoard[i],
            _ENEMY, _FRIEND);
    for (pSq = gEstimatesStart; pSq != gEstimatesEnd; ++pSq)

```

```

*pSq = 0;
for (pSq = gFirstStone; pSq <= gLastStone; ++pSq)
if (OCCUPIED(*pSq)) {
    d = gDirections;
    do {
        pNewSq = pSq + *d;
        if (EMPTY(*pNewSq)) {
            gEstimates[GET_INDEX(pNewSq)] += ESTIMATE_PLUS;
            pNewSq += *d;
            if (EMPTY(*pNewSq)) {
                gEstimates[GET_INDEX(pNewSq)] += ESTIMATE_PLUS;
            }
        }
    } while (++d != gDirectionsEnd);
}

for (pSq = gBoardStart; pSq != gBoardEnd; ++pSq)
if (gEstimates[GET_INDEX(pSq)]) {
    i = GET_INDEX(pSq);
    gScore = gEstimates[i];
    gPreScore[i] = AddStone(-INFINITY, INFINITY, pSq,
        _FRIEND, 0, gCumCapturesFriend,
        gCumCapturesEnemy, gFirstStone,
        gLastStone);
}

for (i=0; i<SEARCH_DEPTH; ++i)
    gKillers[i] = NULL;
bestScore = -INFINITY;
pBestMove = NULL;
pSq = ChooseNextMove();
while (pSq) {
    i = GET_INDEX(pSq);
    gScore = gEstimates[i];
    score = AddStone(-INFINITY, -bestScore, pSq, _FRIEND,
        gStartDepth, gCumCapturesFriend,
        gCumCapturesEnemy, gFirstStone, gLastStone);
    gEstimates[i] = -1; // searched
    if (score > bestScore) {
        bestScore = score;
        pBestMove = pSq;
    }
    pSq = ChooseNextMove();
}

if (bestScore == -INFINITY) {
    for (pSq = gBoardStart; pSq != gBoardEnd; ++pSq)
        if (EMPTY(*pSq) && !gEstimates[GET_INDEX(pSq)]) {
            gScore = 0;
            score = AddStone(-INFINITY, -bestScore, pSq, _FRIEND,
                gStartDepth, gCumCapturesFriend,
                gCumCapturesEnemy, gFirstStone, gLastStone);
            if (score > bestScore) {
                bestScore = score;
                pBestMove = pSq;
            }
        }
    if (bestScore == -INFINITY) {
        DebugStr("\p no move found - Pente");
        return; //no move
    }
}

*pBestMove = _FRIEND;
UPDATE_ENDPOINTS(pBestMove, gFirstStone, gLastStone);
i = GET_INDEX(pBestMove);
move.h = GET_X(i);
move.v = GET_Y(i);
*yourMove = move;
++gMoveNum;
// find captures
*numCaptures = MyFindCaptures(claimCaptures, pBestMove,
    FRIEND, ENEMY);

```

```

gCumCapturesFriend += *numCaptures;
if (gCumCapturesFriend >= 5) {
    *claimVictory = true;
} else {
    *claimVictory = MyFindFive(pBestMove, _FRIEND);
}
}

TermPente
void TermPente(void) { }

ChooseNextMove
short * ChooseNextMove()
{
    short i, *pSq;
    short *found = NULL;
    short high = -INFINITY;
    for (pSq = gBoardStart; pSq != gBoardEnd; ++pSq) {
        i = GET_INDEX(pSq);
        if (gEstimates[i] > 0 && gPreScore[i] > high) {
            found = pSq;
            high = gPreScore[i];
        }
    }
    return found;
}

AddStone
// Alpha-beta search routine
// returns score of how good it is for color
// alpha and beta apply for the opponent after the move
// is made
// gScore is absolute: + for _FRIEND
short AddStone(short alpha, short beta, short *pSq,
    short color, short depth, short capturesFriend,
    short capturesEnemy, short *firstStone,
    short *lastStone)
{
    register short *pNewSq, *d;
    short x, bestScore, t, *pEnd;
    short open, *killer;
    short threats, blocks, vulnerable;
    short opponent = OPPONENT(color);
    short saveScore = gScore;

    START_SAVE;
    threats = blocks = vulnerable = 0;
    d = gDirections;
    do {
        pNewSq = pSq + *d;

        if (*pNewSq == color) { // Next to friend
            if (d <= &gDirections[3]) {
                x = 1;
                open = 0;
                for (pNewSq += *d; *pNewSq == color; pNewSq += *d)
                    ++x;
                if (EMPTY(*pNewSq))
                    open = 1;
                for (pNewSq = pSq + *(d+4); *pNewSq == color;
                    pNewSq += *(d+4))
                    ++x;
                if (EMPTY(*pNewSq))
                    ++open;
            } else {
                t = *(pSq + *(d-4));
                if (t == color)
                    continue;
                x = 1;
                open = 0;
                if (EMPTY(t))
                    open = 1;
                for (pNewSq += *d; *pNewSq == color; pNewSq += *d)

```

```

    ++x;
    if (EMPTY(*pNewSq))
        ++open;
}

switch (x) {
case 1: // 2-in-a-row
    gScore += CHAIN_SCORE2[color][open];
    if (open == 1)
        ++vulnerable;
    break;
case 2: // 3-in-a-row
    gScore += CHAIN_SCORE3[color][open];
    if (open == 2)
        threats += 2; // tria = 2
    break;
case 3: // 4-in-a-row
    gScore += CHAIN_SCORE4[color][open];
    threats += open * 3; // tessera = 6, half-open = 3
    break;
default: // 5-in-a-row (or more)
    gScore += CHAIN_SCORE5[color];
    threats = -99; // game over
    break;
}

} else if (*pNewSq == opponent) { // Next to enemy
    x = 1;
    for (pNewSq += *d; *pNewSq == opponent; pNewSq += *d)
        ++x;

    if (EMPTY(*pNewSq)) {
        blocks += BLOCK_SCORE[x];
    } else if (x == 2 && *pNewSq == color) {
        t = CAPTURE_SCORE;
        if (color != _FRIEND)
            t = -t;
        gScore += t;
        if (color == _FRIEND) {
            if (++capturesFriend >= 5) {
                threats = -99; // game over
            }
        } else { // color == _ENEMY
            if (++capturesEnemy >= 5) {
                threats = -99; // game over
            }
        }
        if (depth) {
            pNewSq = pSq + *d;
            PUSH_SQ(pNewSq);
            *pNewSq = _EMPTY;
            pNewSq += *d;
            PUSH_SQ(pNewSq);
            *pNewSq = _EMPTY;
        }
    }
}

} while (++d != gDirectionsEnd);

if (threats < 0) {
    // Game over
    bestScore = gScore;
    if (color != _FRIEND)
        bestScore = -bestScore;
    goto RESTORE;
}

// Not forced to make a bad move
if (color == _FRIEND) {
    if (gScore < saveScore)
        gScore = saveScore;
} else if (gScore > saveScore)
    gScore = saveScore;

```

```

if (depth) {
    // Add stone
    PUSH_SQ(pSq);
    *pSq = color;
    UPDATE_ENDPOINTS(pSq, firstStone, lastStone);
    --depth;
    gPreviousThreats = threats;
    bestScore = -INFINITY;
    // Killer move?
    killer = gKillers[depth];
    if (killer && EMPTY(*killer)) {
        saveScore = gScore;
        bestScore = AddStone(-beta, -alpha, killer,
            opponent, depth, capturesFriend,
            capturesEnemy, firstStone, lastStone);
        gScore = saveScore;
        if (bestScore > alpha) {
            if (bestScore >= beta) {
                bestScore = -bestScore;
                goto RESTORE;
            }
            alpha = bestScore;
        }
    }
}

pEnd = lastStone + gSE;
if (pEnd >= gBoardEnd)
    pEnd = gBoardEnd - 1;
pSq = firstStone - gSE;
if (pSq < gBoardStart)
    pSq = gBoardStart;
do {
    if (EMPTY(*pSq) && pSq != killer) {
        d = gDirections;
        do {
            pNewSq = pSq + *d;
            if (OCCUPIED(*pNewSq) &&
                (*(pNewSq + *d) == *pNewSq ||
                 *(pSq - *d) == *pNewSq))
                break;
        } while (++d != gDirectionsEnd);
        if (d != gDirectionsEnd) {
            saveScore = gScore;
            t = AddStone(-beta, -alpha, pSq, opponent,
                depth, capturesFriend, capturesEnemy,
                firstStone, lastStone);
            gScore = saveScore;
            if (t > bestScore) {
                bestScore = t;
                if (t > alpha) {
                    if (t >= beta) {
                        gKillers[depth] = pSq;
                        break;
                    }
                    if (depth >= gStartDepth-1)
                        gKillers[depth] = pSq;
                    alpha = t;
                }
            }
        }
    }
} while (++pSq <= pEnd);
if (bestScore == -INFINITY) {
    ++depth;
    goto TERMINAL;
}
bestScore = -bestScore;

} else { // !depth
TERMINAL:
    bestScore = gScore;
    if (color != _FRIEND)
        bestScore = -bestScore;
}

```

```

// Winning threats
if (gPreviousThreats >= 5) {
    bestScore -= THREATS[gPreviousThreats];
} else if (threats >= 5) {
    bestScore += THREATS[threats];
} else if (gPreviousThreats == 4) {
    bestScore -= THREATS[gPreviousThreats];
} else if (threats == 4) {
    bestScore += THREATS[threats];
} else if (gPreviousThreats) {
    bestScore -= THREATS[gPreviousThreats];
} else if (threats) {
    bestScore += THREATS[threats];
}
bestScore += blocks - vulnerable * VULNERABLE;
}

RESTORE:
while (POP) {
    pSq = (short *)TOP;
    *pSq = POP;
}

return bestScore;
}

MyFindCaptures
long MyFindCaptures(Capture capture[], short *pSq,
    short us, short them)
{
    short i, *p1, *p2;
    short myCaptures = 0;
    short *d = gDirections;

    do {
        p1 = pSq + *d;
        if (*p1 == them) {
            p2 = p1 + *d;
            if (*p2 == them && *(p2 + *d) == us) {
                *p1 = *p2 = _EMPTY;
                i = GET_INDEX(p1);
                capture[myCaptures].stone1.h = GET_X(i);
                capture[myCaptures].stone1.v = GET_Y(i);
                i = GET_INDEX(p2);
                capture[myCaptures].stone2.h = GET_X(i);
                capture[myCaptures].stone2.v = GET_Y(i);
                ++myCaptures;
            }
        }
    } while (++d != gDirectionsEnd);
    return myCaptures;
}

MyFindFive
Boolean MyFindFive(short *pSq, short color)
{
    short x, *d, *pNewSq, i;

    d = gDirections;
    i = 0;
    do {
        x = 1;
        for (pNewSq = pSq + *d; *pNewSq == color; pNewSq += *d)
            ++x;
        for (pNewSq = pSq + *(d+4); *pNewSq == color;
            pNewSq += *(d+4))
            ++x;
        if (x >= 5)
            return true;
        ++d;
    } while (++i < 4);
}

```

```
return false;
}
```

SPREAD THE WORD:  [Slashdot](#)  [Digg](#)  [Del.icio.us](#)  [Reddit](#)  [Newsvine](#)

Generate a short URL for this page:

GET LINK

MacTech Magazine. www.mactech.com

Toll Free 877-MACTECH, Outside US/Canada: 805-494-9797

MacTech is a registered trademark of Xplain Corporation. Xplain, "The journal of Apple technology", Apple Expo, Explain It, MacDev, MacDev-1, THINK Reference, NetProfessional, Apple Expo, MacTech Central, MacTech Domains, MacNews, MacForge, and the MacTutorMan are trademarks or service marks of Xplain Corporation. Sprocket is a registered trademark of eSprocket Corporation. Other trademarks and copyrights appearing in this printing or software remain the property of their respective holders. Not responsible for typographical errors.

All contents are Copyright 1984-2011 by Xplain Corporation. All rights reserved. Theme designed by [Icreon](#).